

```

"""
pipeline.py
=====

ORQUESTADOR del proceso completo. Encadena las fases, construye el resumen
ejecutivo y delega la exportacion.

Flujo
-----
    carga -> tipificacion -> validacion del panel -> fase 0 -> fase 1
        -> fase 2 -> fase 3 -> fase 4 (opcional) -> resumen -> Excel

Ninguna metrica se calcula aqui: este modulo solo coordina y consolida.
Los nombres de target / id / tiempo se toman siempre del objeto de
configuracion, nunca de literales.
"""

from __future__ import annotations

import platform
import sys
import time
from datetime import datetime
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

from . import (
    fase0_diagnostico,
    fase1_univariado,
    fase2_bivariado,
    fase3_multivariado,
    fase4_boruta,
    io_utils,
    reporte_excel,
    validaciones,
)
from .config import ConfigPipeline
from .logging_utils import ManejadorMemoria, obtener_logger

LOGGER = obtener_logger("pipeline")

# -----
# Resumen ejecutivo
# -----
def _construir_embudo(
    n_total: int, uni: pd.DataFrame, biv: pd.DataFrame, multi: pd.DataFrame,
    cfg: ConfigPipeline, boruta_meta: dict,
) -> pd.DataFrame:
    """Tabla del embudo: cuantas variables entran, salen y sobreviven por fase."""
    n_cand = int((uni["rol"] == "CANDIDATA").sum()) if not uni.empty else 0
    n_uni = int(uni["flg_seleccionada_univariada"].sum()) if not uni.empty else 0
    n_biv = int((biv["flg_exclusion"] == 0).sum()) if not biv.empty else 0
    n_multi = int(multi["flg_seleccion_final"].sum()) if not multi.empty else 0

    filas = [
        {
            "fase": "0. Dataset original",
            "entran": n_total, "eliminadas": n_total - n_cand, "sobreviven": n_cand,
            "criterio_aplicado": (
                f"Se apartan las columnas de rol (target='{cfg.columna_target}', "
                f"id='{cfg.columna_id}', tiempo='{cfg.columna_tiempo}') "
                f"y {len(cfg.columnas_excluidas)} exclusiones manuales."
            ),
        },
        {
            "fase": "1. Univariado",
            "entran": n_cand, "eliminadas": n_cand - n_uni, "sobreviven": n_uni,

```

```

"criterio_aplicado": (
    f"ceros+nulos >= {cfg.umbral_ceros_nulos_efectivo:.0%} "
    f"({'alterno' if cfg.usar_umbral_alterno else 'principal'}) o baja variacion "
    f"(std<={cfg.umbral_std_minimo:.0e}, CV<={cfg.umbral_cv_minimo:.0e}, "
    f"dominancia>={cfg.umbral_dominancia:.0%}, IQR<={cfg.umbral_iqr_minimo:g})."
),
},
{
    "fase": "2. Bivariado",
    "entran": n_uni, "eliminadas": n_uni - n_biv, "sobreviven": n_biv,
    "criterio_aplicado": (
        f"IV < umbral Y Gini < umbral (ambas condiciones). Umbrales base "
        f"IV={cfg.umbral_iv_minimo}, Gini={cfg.umbral_gini_minimo}"
        + (f", elevados al piso de ruido estadistico (alpha={cfg.alpha_ruido}) cuando este "
            "es mayor. " if cfg.usar_piso_ruido else ". ")
        + f"Score compuesto = {cfg.peso_gini:.0%} Gini + {cfg.peso_iv:.0%} IV, "
        f"normalizacion '{cfg.metodo_normalizacion}'."
    ),
},
{
    "fase": "3. Multivariado",
    "entran": n_biv, "eliminadas": n_biv - n_multi, "sobreviven": n_multi,
    "criterio_aplicado": (
        f"|asociacion| > {cfg.umbral_correlacion}; en cada par se conserva la "
        f"variable con mayor score compuesto."
        + (f" VIF > {cfg.umbral_vif} tambien elimina." if cfg.excluir_por_vif
            else f" VIF > {cfg.umbral_vif} solo se informa.")
    ),
},
{
    "fase": "4. Boruta (opcional)",
    "entran": n_uni if boruta_meta.get("ejecutada") else 0,
    "eliminadas": 0,
    "sobreviven": boruta_meta.get("n_confirmadas", 0) if boruta_meta.get("ejecutada") else 0,
    "criterio_aplicado": (
        f"Motor '{boruta_meta.get('motor', 'n/d')}': {boruta_meta.get('n_confirmadas', 0)} confirmadas, "
        f"{boruta_meta.get('n_tentativas', 0)} tentativas, {boruta_meta.get('n_rechazadas', 0)} rechazadas."
        "NO modifica la seleccion final: se usa como contraste."
        if boruta_meta.get("ejecutada")
        else f"No ejecutada. {boruta_meta.get('observacion', '')}"
    ),
},
{
    "fase": "SELECCION FINAL",
    "entran": n_cand, "eliminadas": n_cand - n_multi, "sobreviven": n_multi,
    "criterio_aplicado": (
        f"Variables que superan las fases 1, 2 y 3. Retencion global: "
        f"{n_multi / n_cand:.1%} de las candidatas."
        if n_cand else "Sin candidatas."
    ),
},
]
return pd.DataFrame(filas)

```

```

def _construir_resumen(
    cfg: ConfigPipeline, df: pd.DataFrame, uni: pd.DataFrame, biv: pd.DataFrame,
    multi: pd.DataFrame, boruta_meta: dict, rep_val: validaciones.ReporteValidacion,
    diagnostico: pd.DataFrame, segundos: float,
) -> pd.DataFrame:
    """Resumen ejecutivo en formato seccion / concepto / valor / comentario."""
    n_total = df.shape[1]
    n_cand = int((uni["rol"] == "CANDIDATA").sum()) if not uni.empty else 0
    n_uni = int((uni["flg_seleccionada_univariada"].sum()) if not uni.empty else 0
    n_biv = int((biv["flg_exclusion"] == 0).sum()) if not biv.empty else 0
    n_multi = int((multi["flg_seleccion_final"].sum()) if not multi.empty else 0

    filas: list[tuple[str, str, Any, str]] = []
    add = lambda s, c, v, o="": filas.append((s, c, v, o)) # noqa: E731

    # --- A. Identificacion de la corrida -----

```

```

add("A. Corrida", "Fecha y hora", datetime.now().strftime("%Y-%m-%d %H:%M:%S"), "Momento de ejecucion.")
add("A. Corrida", "Duracion (segundos)", round(segundos, 2), "Tiempo total del pipeline.")
add("A. Corrida", "Python", sys.version.split()[0], f"{platform.system()} {platform.release()}")
add("A. Corrida", "Semilla", cfg.semilla, "Fija la reproducibilidad de Boruta y del submuestreo.")

# --- B. Definicion de columnas de entrada -----
add("B. Entradas", "ruta_dataset", cfg.ruta_dataset, "Origen de los datos.")
add("B. Entradas", "columna_target", cfg.columna_target, f"Tipo detectado: {rep_val.tipo_target}.")
add("B. Entradas", "columna_id", cfg.columna_id, f"{rep_val.n_entidades} entidades distintas.")
add("B. Entradas", "columna_tiempo", cfg.columna_tiempo, f"{rep_val.n_periodos} periodos distintos.")
add("B. Entradas", "usar_boruta", cfg.usar_boruta,
     "Controla la ejecucion de la fase 4." )
add("B. Entradas", "ruta_salida_excel", cfg.ruta_salida_excel, "Destino de esta bitacora.")
add("B. Entradas", "columnas_excluidas", "", ".join(cfg.columnas_excluidas) or "(ninguna)",
     "Columnas apartadas por decision del usuario, sin evaluar.")

# --- C. Estado del dataset -----
add("C. Dataset", "Filas", df.shape[0], "Observaciones del panel.")
add("C. Dataset", "Columnas totales", n_total, "Incluye las tres columnas de rol.")
add("C. Dataset", "Estructura del panel",
     f"{rep_val.n_entidades} entidades x {rep_val.n_periodos} periodos",
     "Balanceado" if rep_val.panel_balanceado else "Desbalanceado: entidades con distinto numero de periodos.")
add("C. Dataset", "Duplicados en llave id+tiempo", rep_val.duplicados_llave,
     "Debe ser 0. Un panel exige una observacion por entidad y periodo.")
pct_nulos_global = float(df.isna().sum().sum() / max(df.size, 1))
add("C. Dataset", "Nulos globales", f"{pct_nulos_global:.4%}", "Proporcion de celdas vacias del dataset.")

# --- D. Embudo de seleccion -----
add("D. Embudo", "Columnas que entran a evaluacion", n_cand,
     "Total menos target, id, tiempo y exclusiones manuales.")
add("D. Embudo", "Pasan la fase univariada", n_uni,
     f"Se eliminaron {n_cand - n_uni} ({(n_cand - n_uni) / n_cand:.1%} de las candidatas)." if n_cand else "")
add("D. Embudo", "Pasan la fase bivariada", n_biv,
     f"Se eliminaron {n_uni - n_biv} por poder predictivo insuficiente." if n_uni else "")
add("D. Embudo", "Quedan tras la fase multivariada", n_multi,
     f"Se eliminaron {n_biv - n_multi} por redundancia con otra variable mas predictiva." if n_biv else "")
add("D. Embudo", "Boruta ejecutado", "SI" if boruta_meta.get("ejecutada") else "NO",
     boruta_meta.get("observacion", ""))
if boruta_meta.get("ejecutada"):
    add("D. Embudo", "Boruta - confirmadas", boruta_meta.get("n_confirmadas", 0),
         "Importancia significativamente superior a las shadow features.")
    add("D. Embudo", "Boruta - tentativas", boruta_meta.get("n_tentativas", 0),
         "Sin evidencia concluyente en ninguna direccion.")
    add("D. Embudo", "Boruta - rechazadas", boruta_meta.get("n_rechazadas", 0),
         "No superan al ruido de forma significativa.")
add("D. Embudo", "Retencion global", f"{n_multi / n_cand:.1%}" if n_cand else "n/a",
     "Proporcion de candidatas que llega a la seleccion final.")

# --- E. Criterios de exclusion mas importantes -----
if not uni.empty:
    cand = uni[uni["rol"] == "CANDIDATA"]
    add("E. Criterios", f"Eliminadas por ceros+nulos >= {cfg.umbral_ceros_nulos_efectivo:.0%}",
         int(cand["flg_eliminado_ceros"].sum()),
         "Variables sin contenido informativo en casi toda la muestra.")
    add("E. Criterios", "Eliminadas por baja variacion",
         int(cand["flg_eliminado_variacion"].sum()),
         "Constantes, casi constantes o con una categoria dominante.")
    for sub, etiqueta in (
        ("flg_constante", "...de ellas, constantes puras"),
        ("flg_categoria_dominante", f"...con categoria dominante >= {cfg.umbral_dominancia:.0%}"),
        ("flg_cv_bajo", "...con coeficiente de variacion despreciable"),
        ("flg_percentiles_comprimidos", "...con percentiles comprimidos"),
    ):
        if sub in cand.columns:
            add("E. Criterios", etiqueta, int(cand[sub].sum()), "Subcriterio de la fase 1.2.")
if not biv.empty:
    add("E. Criterios", "Eliminadas por IV y Gini por debajo del umbral",
         int(((biv["flg_iv_bajo"] == 1) & (biv["flg_gini_bajo"] == 1)).sum()),
         f"Umbral base IV>={cfg.umbral_iv_minimo}, Gini>={cfg.umbral_gini_minimo}. "
         + ("Elevados al piso de ruido cuando este es mayor." if cfg.usar_piso_ruido else ""))
    if cfg.usar_piso_ruido and "piso_ruido_iv" in biv.columns:

```

```

        piso_tip = biv["piso_ruido_iv"].median()
        add("E. Criterios", "Piso de ruido del IV (mediana)",
            round(float(piso_tip), 4) if pd.notna(piso_tip) else "n/a",
            "IV que alcanzaria una variable ALEATORIA con esta muestra y este numero de bins. "
            "Un IV por debajo no es evidencia de senal.")
        add("E. Criterios", "Variables que NO baten al azar",
            int((biv["flg_supera_ruido"] == 0).sum()) if "flg_supera_ruido" in biv.columns else 0,
            "Su IV y su Gini son compatibles con los de una columna de numeros aleatorios.")
        add("E. Criterios", f"Marcadas con sospecha de fuga (IV>{cfg.umbral_iv_sospechoso})",
            int(biv["flg_sospecha_fuga"].sum()),
            "Requieren revision manual: un IV asi de alto suele indicar informacion del futuro.")
        if "flg_inestable_temporal" in biv.columns:
            add("E. Criterios", f"Marcadas por inestabilidad temporal (PSI>{cfg.umbral_psi})",
                int(biv["flg_inestable_temporal"].sum()),
                "Su distribucion cambia entre periodos del panel.")
    if not multi.empty:
        add("E. Criterios", f"Eliminadas por asociacion > {cfg.umbral_correlacion}",
            int(multi["flg_exclusion_multivariada"].sum()),
            "En cada par redundante se conservo la de mayor score compuesto.")
        add("E. Criterios", f"Marcadas por VIF > {cfg.umbral_vif}",
            int(multi["flg_vif_alto"].sum()) if "flg_vif_alto" in multi.columns else 0,
            "Colinealidad multiple: la variable es explicable por combinacion de otras.")

# --- F. Calidad del dataset (comentario generado) -----
comentarios = _evaluar_calidad(diagnostico, rep_val, cfg, n_cand, n_uni, n_multi)
for i, c in enumerate(comentarios, start=1):
    add("F. Calidad del dataset", f"Observacion {i}", c, "")

# --- G. Conclusiones -----
for i, c in enumerate(_conclusiones(cfg, biv, multi, boruta_meta, n_cand, n_multi), start=1):
    add("G. Conclusiones", f"Conclusion {i}", c, "")

return pd.DataFrame(filas, columns=["seccion", "concepto", "valor", "comentario"])

def _evaluar_calidad(
    diagnostico: pd.DataFrame, rep_val: validaciones.ReporteValidacion,
    cfg: ConfigPipeline, n_cand: int, n_uni: int, n_multi: int,
) -> list[str]:
    """Genera el comentario general de calidad del dataset."""
    obs: list[str] = []
    cand = diagnostico[diagnostico["rol"] == "CANDIDATA"]

    if cand.empty:
        return ["No hubo columnas candidatas que evaluar."]

    pct_nulos_medio = float(cand["pct_nulos"].mean())
    if pct_nulos_medio < 0.01:
        obs.append(f"Cobertura de datos excelente: nulos promedio de {pct_nulos_medio:.2%} por columna.")
    elif pct_nulos_medio < 0.10:
        obs.append(f"Cobertura razonable: nulos promedio de {pct_nulos_medio:.2%} por columna.")
    else:
        obs.append(
            f"Cobertura deficiente: nulos promedio de {pct_nulos_medio:.2%} por columna. "
            "Conviene revisar el proceso de extraccion antes de modelar."
        )

    n_muy_vacias = int((cand["pct_ceros_mas_nulos"] > 0.80).sum())
    if n_muy_vacias:
        obs.append(
            f"{n_muy_vacias} columnas superan el 80% de ceros+nulos (aunque no todas alcancen el "
            f"umbral de {cfg.umbral_ceros_nulos_efectivo:.0%}): son variables de cobertura marginal."
        )

    if not rep_val.panel_balanceado:
        obs.append(
            "El panel esta desbalanceado. Las metricas agrupadas (pooled) dan mas peso a las "
            "entidades con mas periodos observados; conviene contrastarlas con las metricas por periodo."
        )

    if rep_val.n_periodos < 4:
        obs.append(

```

```

        f"Solo hay {rep_val.n_periodos} periodos: las metricas de estabilidad temporal "
        "(PSI, IV por periodo) tienen poca potencia."
    )

if "icc_panel" in cand.columns:
    fijas = cand[cand["icc_panel"] > 0.95]["columna"].tolist()
    if fijas:
        plural = len(fijas) > 1
        obs.append(
            f"{len(fijas)} variable{'s' if plural else ''} "
            f"{'son' if plural else 'es'} practicamente invariante{'s' if plural else ''} en el "
            f"tiempo (ICC>0.95): {'', '.join(fijas[:6])}'{'...' if len(fijas) > 6 else ''}. "
            f"{'Son atributos fijos' if plural else 'Es un atributo fijo'} de la entidad; un "
            f"modelo de efectos fijos {'las' if plural else 'la'} absorberia por completo."
        )

if n_cand:
    tasa = n_multi / n_cand
    if tasa < 0.15:
        obs.append(
            f"Solo el {tasa:.1%} de las candidatas llego a la seleccion final: el dataset tiene "
            "mucha redundancia o mucho ruido. Vale la pena revisar el proceso que genera las variables."
        )
    elif tasa > 0.80:
        obs.append(
            f"El {tasa:.1%} de las candidatas sobrevivio: el conjunto de variables es limpio y poco "
            "redundante, o los umbrales son laxos para este dataset."
        )
    else:
        obs.append(f"Retencion del {tasa:.1%} de las candidatas: un embudo dentro de lo esperable.")

n_dup_nombres = int(diagnostico["columna"].duplicated().sum())
if n_dup_nombres:
    obs.append(f"Atencion: {n_dup_nombres} nombres de columna repetidos.")

return obs

def _conclusiones(
    cfg: ConfigPipeline, biv: pd.DataFrame, multi: pd.DataFrame,
    boruta_meta: dict, n_cand: int, n_multi: int,
) -> list[str]:
    """Conclusiones generales del proceso."""
    c: list[str] = []
    c.append(
        f"De {n_cand} variables candidatas se seleccionaron {n_multi} tras aplicar, en cadena, "
        "criterios de calidad (fase 1), de poder predictivo individual (fase 2) y de no redundancia (fase 3)."
    )
    if not multi.empty and n_multi:
        top = multi[multi["flg_seleccion_final"] == 1].nlargest(5, "score_compuesto")
        nombres = ", ".join(f"{r['columna']} (IV={r['iv']:.3f}, Gini={r['gini']:.3f})"
                             for _, r in top.iterrows())
        c.append(f"Las variables mas predictivas de la seleccion final son: {nombres}.")
    if not biv.empty and int(biv["flg_sospecha_fuga"].sum()):
        sos = biv.loc[biv["flg_sospecha_fuga"] == 1, "columna"].tolist()
        c.append(
            f"Revision obligatoria antes de modelar: {'', '.join(sos[:8])}' "
            f"{'presentan' if len(sos) > 1 else 'presenta'} un IV superior a "
            f"{cfg.umbral_iv_sospechoso}, lo que suele delatar fuga de informacion (una variable "
            "construida con datos posteriores al momento de la prediccion)."
        )
    if boruta_meta.get("ejecutada") and not biv.empty and int(biv["flg_sospecha_fuga"].sum()):
        # Una variable con fuga concentra casi toda la importancia del bosque y
        # deja al resto indistinguible del ruido. Si no se advierte, se leeria
        # el resultado de Boruta como "las demas variables no sirven", cuando lo
        # que ocurre es que estan siendo eclipsadas.
        c.append(
            "ADVERTENCIA sobre la lectura de Boruta: hay variables marcadas con sospecha de fuga. "
            "Una variable con fuga concentra casi toda la importancia del Random Forest y hunde la "
            "de las demas por debajo del umbral de las shadow features, de modo que predictores "
            "legitimos aparecen como 'Rejected'. Elimine primero las variables con fuga y vuelva a "

```

```

        "ejecutar la fase 4 antes de sacar conclusiones de sus rechazos."
    )
    if boruta_meta.get("ejecutada"):
        c.append(
            f"Boruta ({boruta_meta.get('motor')}) confirmo {boruta_meta.get('n_confirmadas', 0)} variables. "
            "Se usa como contraste porque mide importancia CONDICIONAL (en presencia de las demas), "
            "mientras que IV y Gini miden poder MARGINAL: las discrepancias senalan variables que "
            "solo aportan en interaccion o que estan subsumidas por otras."
        )
        c.append(
            "Nota metodologica: en datos de panel las observaciones no son independientes (una misma "
            "entidad aparece en varios periodos), lo que puede optimizar la importancia estimada por el "
            "Random Forest. El veredicto de Boruta debe leerse con esa reserva."
        )
    else:
        c.append("La fase 4 (Boruta) no se ejecuto, por lo que no hay contraste de importancia multivariada.")
    c.append(
        "Recomendacion: validar la seleccion final con una particion fuera de tiempo (out-of-time), "
        "entrenando en los primeros periodos y evaluando en los ultimos. Es la unica forma de comprobar "
        "que el poder predictivo detectado sobrevive al paso del tiempo."
    )
    return c

# -----
# Tablas finales
# -----
def _construir_seleccion_final(
    multi: pd.DataFrame, biv: pd.DataFrame, uni: pd.DataFrame,
    boruta: pd.DataFrame, tipos: dict[str, str],
) -> pd.DataFrame:
    """Lista definitiva de variables para modelado, ordenada por poder predictivo."""
    if multi.empty:
        return pd.DataFrame()

    sel = multi[multi["flg_seleccion_final"] == 1].copy()
    if sel.empty:
        return pd.DataFrame({"aviso": ["Ninguna variable supero las tres fases."]})

    sel["tipo_inferido"] = sel["columna"].map(tipos)
    if not biv.empty:
        ref = biv.set_index("columna")
        for c in ("clasificacion_iv", "metodo_binning", "metodo_gini", "psi_max", "advertencias"):
            if c in ref.columns:
                sel[c] = sel["columna"].map(ref[c])
    if boruta is not None and not boruta.empty:
        mapa = boruta.set_index("columna")["boruta_status"]
        sel["boruta_status"] = sel["columna"].map(mapa).fillna("NO_EVALUADA")

    sel = sel.sort_values("score_compuesto", ascending=False).reset_index(drop=True)
    sel.insert(0, "orden", np.arange(1, len(sel) + 1))

    cols = ["orden", "columna", "tipo_inferido", "iv", "gini", "score_compuesto",
            "clasificacion_iv", "correlacion_maxima", "variable_correlacionada", "vif",
            "psi_max", "boruta_status", "metodo_binning", "metodo_gini", "advertencias"]
    return sel[[c for c in cols if c in sel.columns]]

def _construir_descartadas(
    uni: pd.DataFrame, biv: pd.DataFrame, multi: pd.DataFrame,
) -> pd.DataFrame:
    """Trazabilidad completa de cada variable descartada: donde y por que."""
    filas = []

    if not uni.empty:
        for _, f in uni[(uni["rol"] == "CANDIDATA") & (uni["flg_seleccionada_univariada"] == 0)].iterrows():
            filas.append({
                "columna": f["columna"], "fase_de_salida": "1. Univariado",
                "criterio": f["decision_univariada"],
                "motivo": "; ".join(x for x in (f.get("motivo_ceros"), f.get("motivo_variacion")) if x),
                "iv": np.nan, "gini": np.nan, "score_compuesto": np.nan,
            })

```

```

    })
    if not biv.empty:
        for _, f in biv[biv["flg_exclusion"] == 1].iterrows():
            filas.append({
                "columna": f["columna"], "fase_de_salida": "2. Bivariado",
                "criterio": "Poder predictivo insuficiente",
                "motivo": f.get("motivo_exclusion_bivariada", ""),
                "iv": f.get("iv", np.nan), "gini": f.get("gini", np.nan),
                "score_compuesto": f.get("score_compuesto", np.nan),
            })
    if not multi.empty:
        for _, f in multi[multi["flg_exclusion_multivariada"] == 1].iterrows():
            filas.append({
                "columna": f["columna"], "fase_de_salida": "3. Multivariado",
                "criterio": "Redundancia / colinealidad",
                "motivo": f.get("motivo_exclusion_multivariada", ""),
                "iv": f.get("iv", np.nan), "gini": f.get("gini", np.nan),
                "score_compuesto": f.get("score_compuesto", np.nan),
            })

    return pd.DataFrame(filas) if filas else pd.DataFrame(
        {"aviso": ["Ninguna variable fue descartada."]}
    )

def _exportar_dataset_final(
    df: pd.DataFrame, cfg: ConfigPipeline, variables_seleccionadas: list[str],
) -> str | None:
    """Exporta id + tiempo + target + solo las variables que superaron las
    fases 1, 2 y 3, en un unico archivo listo para modelar.

    Se genera SIEMPRE con las columnas de rol como primeras tres columnas
    (en ese orden: id, tiempo, target), de modo que el archivo sea legible
    de inmediato y quede claro cual es la llave del panel y cual el target.

    No se incluyen columnas que no superaron la seleccion, ni siquiera para
    referencia: ese detalle ya vive en la bitacora Excel (`06b_Descartadas`).
    Este archivo tiene un unico proposito, ser el insumo directo de un
    modelo, y mezclar variables descartadas lo contaminaria.
    """
    if not cfg.exportar_dataset_final:
        LOGGER.info("Exportacion del dataset final OMITIDA (exportar_dataset_final=False).")
        return None

    if not variables_seleccionadas:
        LOGGER.warning(
            "Ninguna variable supero las tres fases: no se exporta dataset final "
            "(quedaria solo con id, tiempo y target)."
        )
        return None

    columnas_rol = [cfg.columna_id, cfg.columna_tiempo, cfg.columna_target]
    columnas_finales = columnas_rol + [c for c in variables_seleccionadas if c not in columnas_rol]
    df_final = df[columnas_finales].copy()

    ruta = cfg.ruta_dataset_final_efectiva
    ruta.parent.mkdir(parents=True, exist_ok=True)

    if cfg.formato_dataset_final == "parquet":
        df_final.to_parquet(ruta, index=False)
    else:
        df_final.to_csv(ruta, index=False, sep=cfg.csv_sep, encoding=cfg.csv_encoding)

    LOGGER.info(
        "Dataset final exportado: %s (%d filas x %d columnas: id+tiempo+target + %d variables seleccionadas).",
        ruta.resolve(), df_final.shape[0], df_final.shape[1], len(variables_seleccionadas),
    )
    return str(ruta.resolve())

```

# -----

```

# Punto de entrada del pipeline
# -----
def ejecutar(
    cfg: ConfigPipeline,
    handler_log: ManejadorMemoria | None = None,
    reporte_bootstrap: Any = None,
) -> dict[str, Any]:
    """Ejecuta el pipeline completo y exporta la bitacora.

    Returns
    -----
    dict
        Todas las tablas intermedias, la ruta del Excel generado y las listas
        de variables seleccionadas y descartadas.
    """
    t0 = time.perf_counter()

    LOGGER.info("#" * 78)
    LOGGER.info("# PIPELINE DE SELECCION DE VARIABLES PARA DATOS DE PANEL")
    LOGGER.info("# target='%s' | id='%s' | tiempo='%s' | usar_boruta=%s",
                cfg.columna_target, cfg.columna_id, cfg.columna_tiempo, cfg.usar_boruta)
    LOGGER.info("#" * 78)

    # === Carga y tipificacion =====
    df = io_utils.cargar_dataset(cfg)
    df, tipos = io_utils.tipificar_dataset(df, cfg)

    # === Validacion estructural del panel =====
    rep_val = validaciones.validar_panel(df, cfg)

    # === FASE 0 =====
    diag = fase0_diagnostico.ejecutar(df, cfg, tipos, rep_val)

    # === FASE 1 =====
    uni = fase1_univariado.ejecutar(diag["diagnostico"], cfg)
    sobrevivientes_1 = fase1_univariado.obtener_sobrevivientes(uni)

    # === FASE 2 =====
    biv, tablas_woe = fase2_bivariado.ejecutar(
        df, cfg, tipos, sobrevivientes_1, rep_val.tipo_target
    )
    sobrevivientes_2 = fase2_bivariado.obtener_sobrevivientes(biv)

    # === FASE 3 =====
    multi, matriz, pares = fase3_multivariado.ejecutar(df, cfg, tipos, sobrevivientes_2, biv)
    seleccion_final = fase3_multivariado.obtener_seleccion_final(multi)

    # === FASE 4 (opcional) =====
    # Se ejecuta sobre los sobrevivientes de la FASE 1, no sobre la seleccion
    # final: asi Boruta puede opinar tambien sobre las variables que las fases
    # 2 y 3 descartaron, que es justamente donde el contraste es informativo.
    boruta, boruta_meta = fase4_boruta.ejecutar(
        df, cfg, tipos, sobrevivientes_1, rep_val.tipo_target, biv, multi
    )

    # === Consolidacion =====
    segundos = time.perf_counter() - t0
    resumen = _construir_resumen(cfg, df, uni, biv, multi, boruta_meta, rep_val,
                                diag["diagnostico"], segundos)
    embudo = _construir_embudo(df.shape[1], uni, biv, multi, cfg, boruta_meta)
    tabla_final = _construir_seleccion_final(multi, biv, uni, boruta, tipos)
    descartadas = _construir_descartadas(uni, biv, multi)

    woe_concat = (
        pd.concat(tablas_woe.values(), ignore_index=True) if tablas_woe else pd.DataFrame()
    )
    log_df = pd.DataFrame(handler_log.registros) if handler_log else pd.DataFrame()
    deps_df = (
        pd.DataFrame(reporte_bootstrap.a_filas()) if reporte_bootstrap is not None else pd.DataFrame()
    )

```



```

resultados: dict[str, Any] = {
    "cfg": cfg,
    "cfg_dict": cfg.a_dict(),
    "resumen": resumen,
    "embudo": embudo,
    "diagnostico": diag["diagnostico"],
    "general": diag["general"],
    "target_por_periodo": diag["target_por_periodo"],
    "exclusion_temprana": diag["exclusion_temprana"],
    "validacion": rep_val.a_dataframe(),
    "univariado": uni,
    "bivariado": biv,
    "multivariado": multi,
    "matriz_asociacion": matriz,
    "pares_redundantes": pares,
    "tablas_woe": woe_concat,
    "boruta": boruta,
    "boruta_meta": boruta_meta,
    "seleccion_final": tabla_final,
    "descartadas": descartadas,
    "parametros": pd.DataFrame(cfg.a_filas()),
    "dependencias": deps_df,
    "variables_seleccionadas": seleccion_final,
    "segundos": segundos,
}

# === Exportacion (capa separada) =====
# El log se vuelca justo antes de exportar para que la hoja de bitacora
# incluya todo lo ocurrido hasta este punto.
if handler_log:
    resultados["log"] = pd.DataFrame(handler_log.registros)
ruta = reporte_excel.exportar(resultados, cfg.ruta_salida_excel)
resultados["ruta_excel"] = str(ruta)

# Dataset "listo para modelar": id + tiempo + target + solo lo seleccionado.
resultados["ruta_dataset_final"] = _exportar_dataset_final(df, cfg, seleccion_final)

LOGGER.info("#" * 78)
LOGGER.info("# PROCESO COMPLETADO en %.2f segundos", segundos)
LOGGER.info("# Variables seleccionadas: %d -> %s", len(seleccion_final),
            ", ".join(seleccion_final[:15]) + ("..." if len(seleccion_final) > 15 else ""))
LOGGER.info("# Bitacora: %s", ruta)
if resultados["ruta_dataset_final"]:
    LOGGER.info("# Dataset final: %s", resultados["ruta_dataset_final"])
LOGGER.info("#" * 78)

return resultados

```